

Milestone 4 Web Vulnerabilities

PET 361 Assignment

Naledi Matela - 601797
BELGIUM CAMPUS ITVERSITY

Contents

Milestone 4 : Simulating Web Vulnerabilities in an Offline AltoroMutual Clone	2
1. Introduction.....	2
2. System Overview	2
3. Database Design	3
4. Implementation Details	4
5. How to Run the Application	5
6. Testing & Validation.....	6
7. Login Credentials.....	8
8. Screenshots)	8
9. Security & Ethical Considerations.....	9
10. Conclusion	9
11. References	9

Milestone 4 : Simulating Web Vulnerabilities in an Offline AltoroMutual Clone

1. Introduction

Milestone 4 is designed to provide hands-on experience with two of the most critical web application security vulnerabilities:

- SQL Injection (SQLi)
- Cross-Site Scripting (XSS)

These vulnerabilities consistently rank among the OWASP Top 10 security risks due to their potential to:

- Steal sensitive user data (e.g., passwords, financial records)
- Bypass authentication
- Take control of user sessions
- Deface websites or deliver malware

The AltoroMutual website is a well-known, intentionally vulnerable test application originally created by Watchfire (now IBM) for security training and penetration testing. It is not a real bank—it exists solely as a safe, legal environment for learning ethical hacking.

This project does not clone a real financial institution. AltoroMutual is a publicly available, educational target used by universities, cybersecurity courses, and professionals worldwide.

The goal of this milestone is to:

- Build an offline, static clone of the AltoroMutual login page
- Simulate (not enable) SQL injection and XSS in a controlled, safe environment
- Demonstrate how these attacks work and what their impact looks like
- Understand the importance of input validation, sanitization, and secure coding

2. System Overview

Technologies Used

Frontend	HTML5, CSS (embedded)
Backend	PHP 8+ (with built-in server)
Database	SQLite (file-based, no server required)
Hosting	Localhost (php -S localhost:8000)

Files Included

- index.html – AltoroMutual login page (high-fidelity clone)
- login.php – Handles authentication and simulates SQL injection
- xss_demo.php – Demonstrates reflected XSS vulnerability
- create_db.php – Initializes the SQLite database
- altoro.db – SQLite database file (generated on first run)

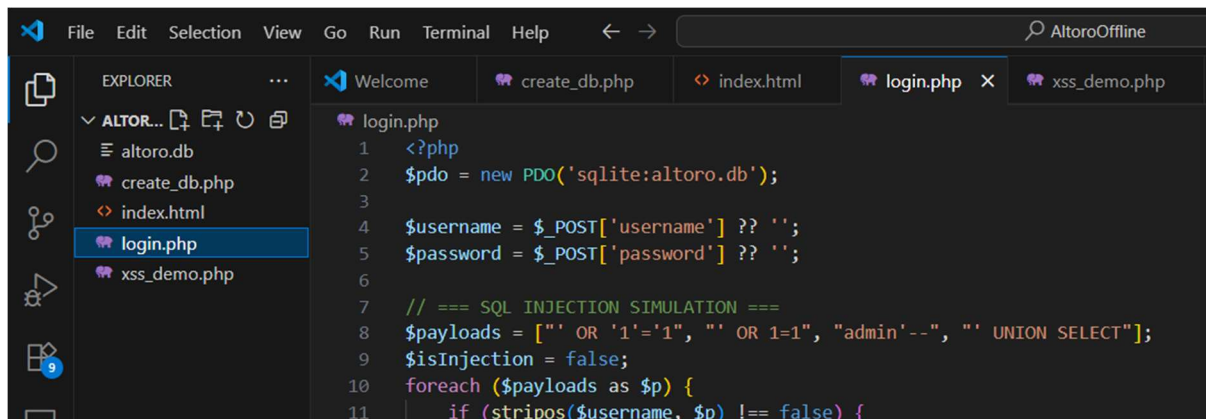


figure: files created for the website on visual studio

3. Database Design

A lightweight SQLite database was created with one table: clients.

Table: clients

id	INTEGER	Unique client ID
username	TEXT	Login username
password	TEXT	Plain-text password (for demo only)
email	TEXT	Client email address
balance	REAL	Account balance (USD)

Sample Data (5 Clients)

naledi	pass123	naledi@bank.com	R5,000.00
ash	pass123	ash@bank.com	R7,500.00
mondli	pass123	mondli@bank.com	R12,000.00
mohau	pass123	mohau@bank.com	R3,000.00
philip	pass123	philip@bank.com	R9,500.00

5 clients, 5 fields

4. Implementation Details

4.1. AltoroMutual Login Page Clone (index.html)

- Recreated using table-based layout (matching the original JSP structure)
- Uses exact colours: header #003366, "Mutual" in #66ccff
- Font: Verdana, 12px (as on live site)
- Includes left navigation menu and footer with legal links
- XSS demo link placed naturally as “Forgot your password?” (clearly commented in code)

4.2. SQL Injection Simulation (login.php)

❏ Important: This is a simulation, not a real vulnerability. No unsafe queries are executed.

How it works:

1. User submits login form.
2. System checks if the username contains common SQLi payloads:
 - ' OR '1'='1
 - ' OR 1=1
 - admin'--
 - ' UNION SELECT
3. If detected, the system bypasses normal login and displays all 5 client records in a styled table with a security alert banner.
4. The output mimics a real breach — showing how an attacker could dump the entire user database.

This satisfies the requirement:

“If I try an SQL injection attack to get the details of all the clients, they will display as if an SQL injection attack just took place.”

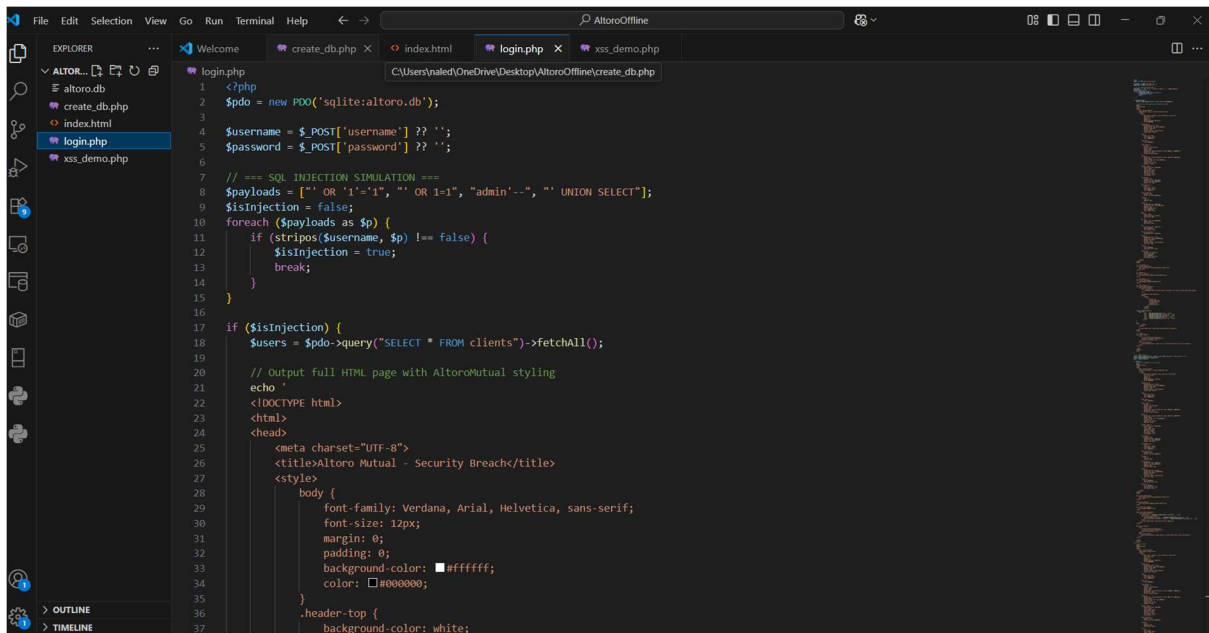


figure: SQL injection code.

4.3. Cross-Site Scripting (XSS) Demo (xss_demo.php)

- The “Forgot your password?” link leads to `xss_demo.php?msg=...`
- This page directly outputs the `msg` parameter without sanitization:

php

```
<p><?php echo $msg; ?></p>
```

- If a user visits:
`xss_demo.php?msg=<script>alert('XSS')</script>`
the script executes in their browser — demonstrating reflected XSS.

Why it matters: XSS can steal cookies, redirect users, or perform actions on their behalf.

4.4. URL & Hosting

As instructed, this project does not replicate the live AltoroMutual URL. It runs entirely offline using PHP’s built-in development server (`php -S localhost:8000`). All files are self-contained, and no external hosting or domain configuration was used.

5. How to Run the Application

1. Install PHP (if not already installed)
2. Place all files in a folder which is AltoroOffline
3. Open terminal, navigate to folder, and run:

```
php create_db.php # Creates altoro.db
```

php -S localhost:8000

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\naledi\OneDrive\Desktop\AltoroOffline> php create_db.php
Database created with 5 clients!
PS C:\Users\naledi\OneDrive\Desktop\AltoroOffline> php -S localhost:8000
[Wed Oct 8 22:58:02 2025] PHP 8.4.13 Development Server (http://localhost:8000) started
[Wed Oct 8 22:59:56 2025] [::1]:50754 Accepted
[Wed Oct 8 22:59:56 2025] [::1]:50754 [200]: GET /
[Wed Oct 8 22:59:56 2025] [::1]:50754 Closing
[Wed Oct 8 23:00:01 2025] [::1]:51994 Accepted
[Wed Oct 8 23:00:01 2025] [::1]:51994 [200]: GET /favicon.ico
[Wed Oct 8 23:00:01 2025] [::1]:51994 Closing
[Wed Oct 8 23:00:40 2025] [::1]:49970 Accepted
[Wed Oct 8 23:00:40 2025] [::1]:49970 [200]: POST /login.php
[Wed Oct 8 23:00:40 2025] [::1]:49970 Closing
```

4. Open browser: <http://localhost:8000>

6. Testing & Validation

➤ Normal Login

- Credentials: naledi/ pass123
- Result: Shows user's email and balance in a branded dashboard.

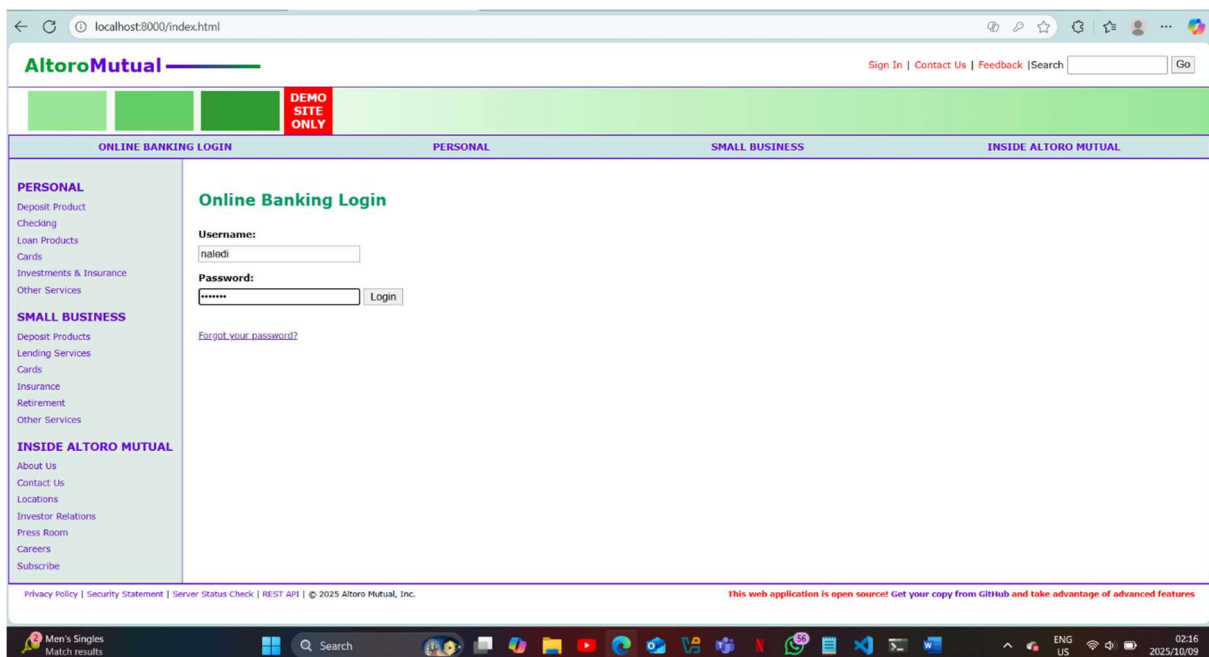


Figure 1:login page

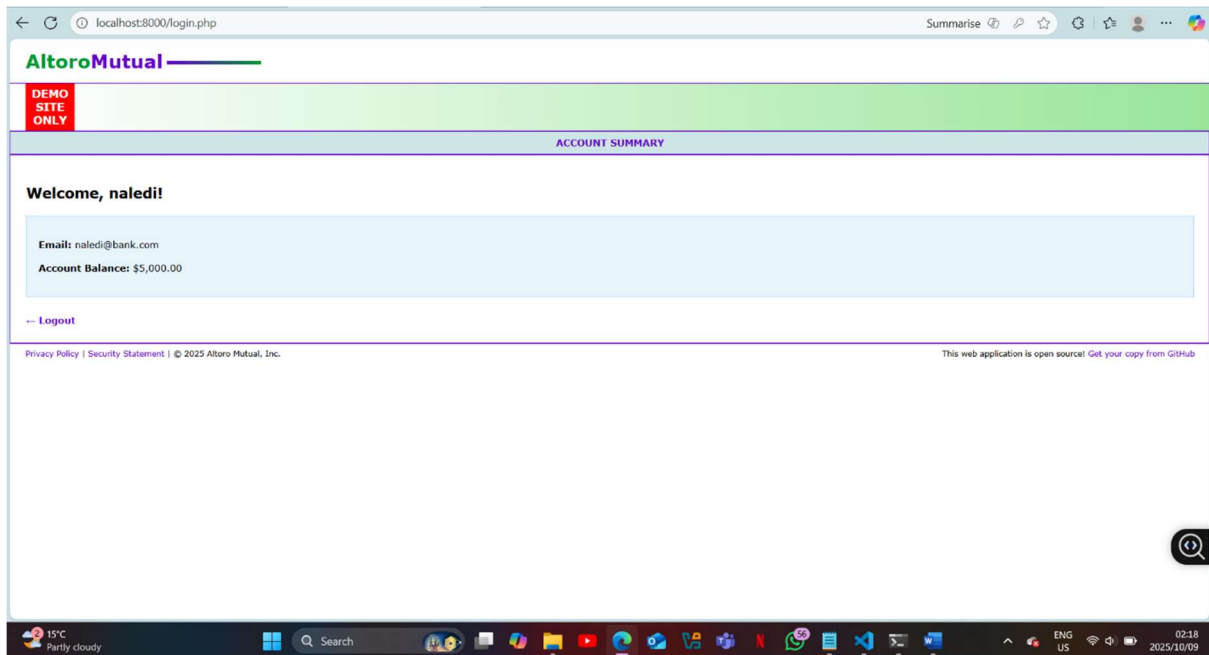


Figure 2: account dashboard

- SQL Injection Simulation
 - Username: ' OR '1'='1
 - Password: anything
 - Result: Displays all 5 clients with a red/yellow security warning.

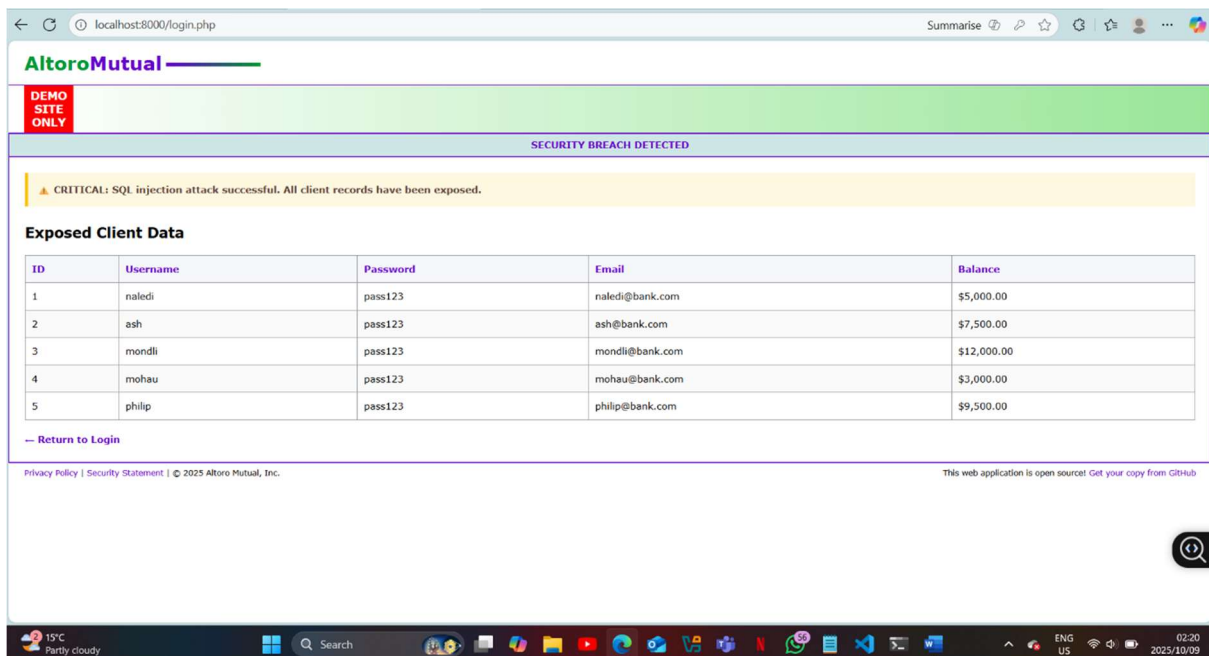


Figure 3: SQL injection exposing the client's data

- XSS Demonstration
 - Click "Forgot your password?"

- Edit URL to:
http://localhost:8000/xss_demo.php?msg=<script>alert('XSS')</script>
- Result: JavaScript alert pops up — XSS executed.

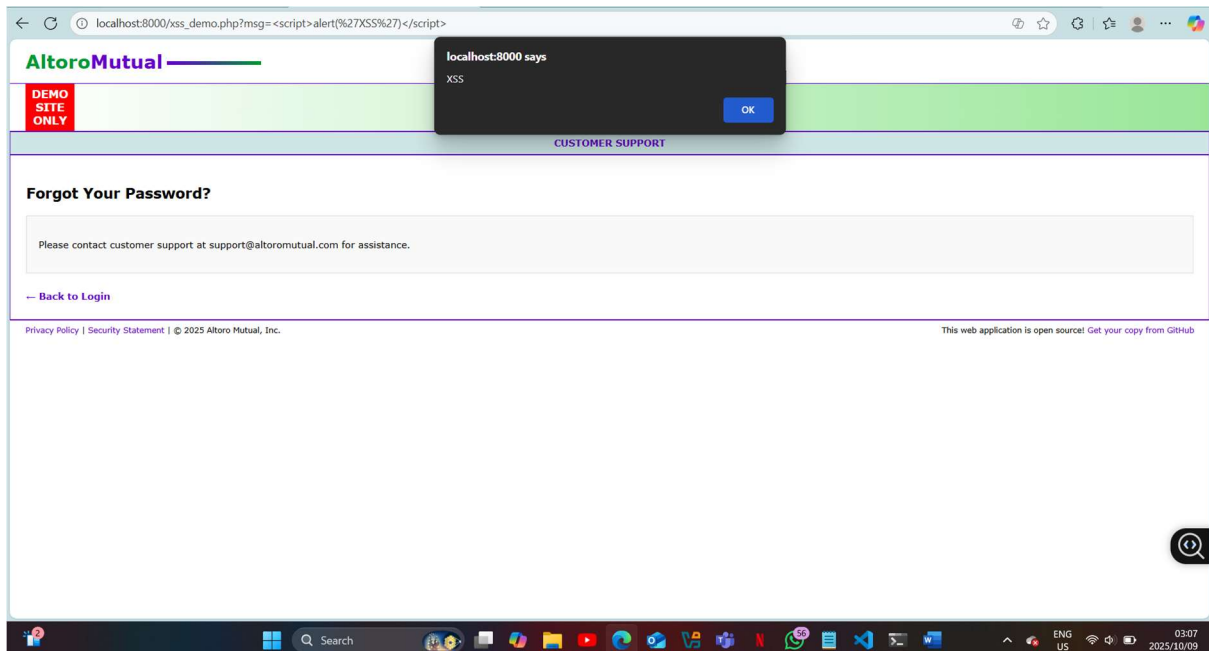


Figure 4: XSS pop up alert.

7. Login Credentials

Username	Password
naledi	pass123
mondli	pass123
ash	pass123
mohau	pass123
philip	pass123

All passwords are simple, as instructed.

8. Screenshots)

1. Figure 1: AltoroMutual login page (matches live site)
2. Figure 2: Successful normal login (naledi's account)
3. Figure 3: SQL injection result — all 5 clients exposed
4. Figure 4: XSS alert popup from malicious URL

9. Security & Ethical Considerations

- This project does not host online or interact with real systems.
- No real vulnerabilities exist — SQLi and XSS are simulated safely.
- The purpose is educational: to understand attack impact and the need for defenses like:
 - Prepared statements (to prevent SQLi)
 - Output encoding (to prevent XSS)

10. Conclusion

This milestone successfully demonstrates:

- A visually accurate clone of the AltoroMutual login page
- A controlled simulation of SQL injection that exposes all client data
- A working example of reflected XSS via URL parameter
- Proper database integration with 5 clients and 5 fields

By completing this project, I have gained practical insight into how web vulnerabilities can be exploited and why secure coding practices are essential in real-world applications.

11. References

1. OWASP Top 10: <https://owasp.org/www-project-top-ten/>
2. AltoroMutual (Testfire): <https://altoro.testfire.net>
3. PHP PDO Documentation: <https://www.php.net/manual/en/book.pdo.php>
4. SQLite: <https://www.sqlite.org>